Mocking the AI responses allows for faster and more controlled testing by avoiding the overhead of calling the actual LLM. However, it's important to note that this method may not reveal issues related to the real-world behavior of the AI model but rather tests any issues with the logic around it.

SEMANTIC SIMILARITY SCORING

Here the main idea is that instead of exact matching, we use semantic similarity measures to compare outputs to expected results.

Listing 5.3 Testing semantic similarity scoring

```
import { generateText } from "ai";
import stringComparison from 'string-comparison'

function semanticSimilarity(text1, text2) {
  let cos = stringComparison.cosine
  return cos.similarity(text1, text2);
}

test("generated text is semantically similar to expected output", async () => {
  generateText.mockResolvedValue(
    "AI greatly influences modern life in many ways."
  );

  const expected = "AI has a significant impact on
modern society.";
  const generated = await generateText(
    "Describe the role of AI in today's world."
  );

  const similarity =
semanticSimilarity(expected, generated);
  expect(similarity).toBeGreaterThan(0.7);
});
```

Imports necessary functions: generateText for AI text generation and string comparison for comparing text similarity

Defines an expected output to compare against the AI-generated text

Calculates the semantic similarity between the expected and generated texts

Asserts that the similarity score is above a threshold (0.7), indicating high semantic similarity

The generated text is then compared to the expected output using the `cosine .simularity` function, which calculates the cosine similarity between them. Finally, the test asserts that the similarity score is above a threshold of 0.7, indicating that the AI-generated text is semantically close to the expected result. Of course, this threshold can be adjusted based on your test criteria.

This approach, although not always bulletproof, allows for more flexible and meaningful testing of AI-generated content, as it evaluates the quality of the output based on semantic understanding rather than exact word matching. This is particularly useful in scenarios where the exact wording may vary but the underlying meaning must remain consistent.

CONSTRAINED VALIDATION

In this kind of testing, we mainly focus on testing that the response from the LLM adheres to any limitations or rules we provided in the prompt. See the following listing, for an example.